Lesson 9: 姿勢推定

Attitude Estimation

StampFly Workshop

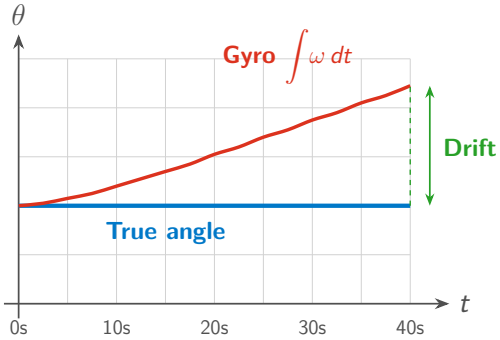
今日のゴール / Today's Goal

目標

相補フィルタを実装し、ESKF の推定値と比較する

- ジャイロ積分のドリフト問題を理解
- 相補フィルタで加速度センサとジャイロを融合
- Teleplot で CF vs ESKF をリアルタイム比較

ジャイロドリフト問題 / Gyro Drift Problem

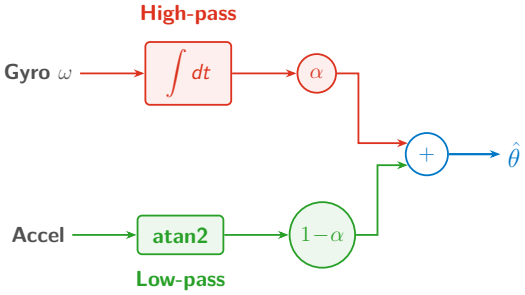


- ジャイロは**角速度** ω を測定
- 角度 $\theta = \int \omega \, dt$（積分が必要）
- バイアスが蓄積 → ドリフト増大

問題

ジャイロ単体では長時間の姿勢推定が不可能

相補フィルタ / Complementary Filter



数式

$$\hat{\theta}_k = \alpha\,(\hat{\theta}_{k-1} + \omega\,\Delta t) + (1-\alpha)\,\theta_{\mathrm{accel}}$$

- $\alpha = 0.98$（ジャイロ 98% + 加速度 2%）
- ジャイロ: 短期精度◎ / 長期ドリフト☒
- 加速度: 長期安定◎ / 振動ノイズ☒

姿勢推定 API

関数	説明	単位
`gyro_x/y/z()`	角速度（Roll/Pitch/Yaw）	rad/s
`accel_x/y/z()`	加速度（X/Y/Z）	m/s^2
`estimated_roll/pitch/yaw()`	ESKF 推定姿勢角	rad
`print(fmt, ...)`	シリアル出力（Teleplot 対応）	—

実習で使う組み合わせ

ジャイロ + 加速度 → 相補フィルタで姿勢を自作

`estimated_roll()` → ESKF の推定値と比較

実習: 相補フィルタ + Teleplot / Hands-on

```cpp
#include "workshop_api.hpp"
#include <cmath>
static float cf_roll = 0.0f, cf_pitch = 0.0f;
void setup() { ws::print("Lesson 9: Attitude Estimation"); }
void loop_400Hz(float dt) {
    float gx = ws::gyro_x(), gy = ws::gyro_y();
    float ax = ws::accel_x(), ay = ws::accel_y(),
          az = ws::accel_z();
    float accel_roll  = atan2f(ay, az);
    float accel_pitch = atan2f(-ax, az);
    constexpr float alpha = 0.98f;
    cf_roll  = alpha*(cf_roll + gx*dt) + (1-alpha)*accel_roll;
    cf_pitch = alpha*(cf_pitch + gy*dt) + (1-alpha)*accel_pitch;
    // Teleplot output (VSCode Teleplot extension)
    ws::print(">cf_roll:%.1f", cf_roll*57.3f);
    ws::print(">eskf_roll:%.1f", ws::estimated_roll()*57.3f);
}
```

user_code.cpp (excerpt)

Teleplot によるリアルタイム可視化

Teleplot フォーマット

>変数名:値 をシリアルに出力するだけ

```
// Teleplot format: >name:value
ws::print(">cf_roll:%.2f", cf_roll * 57.3f);
ws::print(">cf_pitch:%.2f", cf_pitch * 57.3f);
ws::print(">eskf_roll:%.2f",
          ws::estimated_roll() * 57.3f);
```

セットアップ

1. VSCode 拡張: `alexnesnes.teleplot` をインストール
2. `sf monitor` でシリアル接続
3. Teleplot パネルで自動グラフ化

注意: 400 Hz 全 tick で出力するとシリアル帯域に負荷。4 tick 毎（100 Hz）にデシメーション推奨

Madgwick フィルタ / Madgwick Filter

勾配降下法によるクォータニオン推定

チューニングパラメータが β の **1つだけ** — シンプルかつ高精度

- 状態: $\mathbf{q} = [q_0,\ q_1,\ q_2,\ q_3]$
- 予測: $\dot{\mathbf{q}} = \frac{1}{2}\,\mathbf{q} \otimes \boldsymbol{\omega}$
- 補正勾配:
  $\nabla f = \mathbf{J}^T(\hat{\mathbf{q}},\ \hat{\mathbf{d}})\ \mathbf{f}(\hat{\mathbf{q}},\ \hat{\mathbf{d}})$

更新則

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \Delta t\left(\frac{1}{2}\,\mathbf{q}_t \otimes \boldsymbol{\omega} - \beta\,\frac{\nabla f}{|\nabla f|}\right)$$

$\mathbf{f}$: 加速度/磁気の目的関数、$\mathbf{J}$: そのヤコビアン、β: 補正ゲイン（≈ 0.04）

EKF（拡張カルマンフィルタ）

非線形カルマンフィルタの5ステップ

状態ベクトル例: $\mathbf{x} = [\phi,\ \theta,\ \psi,\ b_{gx},\ b_{gy},\ b_{gz}]^T$

ステップ	数式	意味
状態予測	$\hat{\mathbf{x}}^- = f(\hat{\mathbf{x}}, \mathbf{u})$	ジャイロで姿勢を積分
共分散予測	$\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^T + \mathbf{Q}$	不確かさの伝播
カルマンゲイン	$\mathbf{K} = \mathbf{P}^-\mathbf{H}^T(\mathbf{H}\mathbf{P}^-\mathbf{H}^T + \mathbf{R})^{-1}$	予測と観測の信頼度配分
状態更新	$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K}(\mathbf{z} - h(\hat{\mathbf{x}}^-))$	観測で補正
共分散更新	$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$	不確かさを縮小

ESKF（誤差状態カルマンフィルタ）

StampFly で実際に使われている 15 状態 ESKF

名目状態 + 誤差状態の分離アーキテクチャ

誤差状態ベクトル（15 states）:

$$\delta\mathbf{x} = [\delta\mathbf{p},\ \delta\mathbf{v},\ \delta\boldsymbol{\theta},\ \delta\mathbf{b}_g,\ \delta\mathbf{b}_a]^T$$

- 予測: 名目状態はジャイロ/加速度で直接積分
- 誤差共分散のみ伝播

利点: 誤差状態は常に小さい → 線形化が正確

観測更新:

- 気圧 → 高度補正
- ToF → 対地高度補正
- 磁気 → ヨー補正
- 光学フロー → 速度補正

推定手法の比較 / Estimator Comparison

	相補フィルタ	Madgwick	EKF	ESKF
計算量	極小	小	中	中〜大
パラメータ数	1（α）	1（β）	$\mathbf{Q}, \mathbf{R}$ 行列	$\mathbf{Q}, \mathbf{R}$ 行列
推定対象	Roll/Pitch	R/P/Yaw	姿勢+バイアス	位置/速度/姿勢/バイアス
使用センサ	Gyro+Accel	Gyro+Accel+Mag	Gyro+Accel+Mag	全センサ（6 種）
精度	○	◎	◎	◎◎

StampFly のデフォルト: ESKF（15 状態）を使用。このレッスンでは相補フィルタを自作し、ESKF と比較して理解を深める。

チェックポイント / Checkpoint

確認事項

- □ 手で傾けると CF のロール・ピッチが変化する
- □ CF と ESKF の値が概ね一致する（Teleplot で確認）
- □ α を変えて応答の違いを観察
- □ Madgwick / EKF / ESKF の特徴を説明できる

次のレッスン

Lesson 10: API 総覧とアプリケーション開発